

Informe de pruebas

Proyecto: Planificador inteligente de rutas de TransMilenio (Bogotá)

Curso: Inteligencia Artificial, Universidad Iberoamericana

Autor: DanielLopezB2 (proyecto individual)

Repositorio: <https://github.com/DanielLopezB2/inteligencia-artificial-actividad-2>

1. Objetivo

Verificar que el planificador cumple lo que promete: cargar una base de conocimiento validada, inferir hechos con reglas lógicas, construir el grafo y encontrar la ruta de menor costo con A*, con desempate por menos transbordos. Todas las salidas de este informe fueron generadas ejecutando el código real.

2. Entorno

Elemento	Valor
Sistema operativo	macOS (Darwin)
Python	3.14.7
pytest	9.1.1
Dependencias del programa	ninguna (solo biblioteca estándar)

3. Estrategia

- **TDD:** cada feature se desarrolló escribiendo primero las pruebas, viéndolas fallar y luego implementando. La prueba en rojo no queda en el historial de Git, porque cada commit agrupa pruebas y código ya en verde.
- **Una rama por feature:** feat/knowledge-base, feat/graph-builder, feat/astar-search, feat/cli.
- **Integración con merge --no-ff**, para conservar el historial de cada feature.
- **Compuerta antes de avanzar:** pruebas automáticas en verde y una prueba funcional del CLI o del módulo antes de empezar el siguiente feature.

4. Resultados de `python -m pytest -v`

Total: 50 pruebas, 50 aprobadas, 0 fallidas (0.21 s).

Módulo	Pruebas	Qué cubre
tests/test_knowledge_base.py	20	Motor de inferencia (reglas, joins, !=, punto fijo, no muta la entrada), query, adyacencia simétrica, estaciones de transbordo, integridad de network.json, validación de datos (estación o línea desconocida, minutos no positivos).
tests/test_graph.py	8	Estaciones presentes, aristas simétricas con los mismos minutos, vecinos de Ricaurte, transbordos, coordenadas, conectividad, orden determinista, estación aislada.
tests/test_search.py	9	Ruta sin transbordos, ruta con varios transbordos consistente, optimalidad en todos los pares, simetría del costo, desempate por menos transbordos, efecto de la penalización, destino inalcanzable, estación desconocida, origen igual a destino.

Módulo	Pruebas	Qué cubre
tests/test_cli.py	13	Normalización de tildes y mayúsculas, sugerencias, formato de tramos y totales, ruta, misma estación, estación desconocida, --list, --penalty (y rechazo de negativos), modo interactivo, ejecución desde otro directorio.
Total	50	

5. Pruebas funcionales (CLI real)

Cada caso se ejecutó desde la raíz del repositorio.

F1. Ruta en una sola línea

```
$ python -m src.cli "Portal Américas" "Ricaurte"
Route: Portal Américas -> Ricaurte
Ride Americas: Portal Américas -> Ricaurte (4 stops, 17 min)
Ride time: 17 min | Transfers: 0 | Total cost: 17.0 (penalty 5.0 min per transfer)
```

Veredicto: correcto. Sin transbordos, costo igual al tiempo de viaje.

F2. Ruta con varios transbordos

```
$ python -m src.cli "Portal Américas" "Calle 100"
Route: Portal Américas -> Calle 100
Ride Americas: Portal Américas -> Ricaurte (4 stops, 17 min)
Transfer at Ricaurte: Americas -> NQS
Ride NQS: Ricaurte -> Centro Memoria (1 stop, 5 min)
Transfer at Centro Memoria: NQS -> Calle 26
Ride Calle 26: Centro Memoria -> Calle 26 (1 stop, 2 min)
Transfer at Calle 26: Calle 26 -> Caracas
Ride Caracas: Calle 26 -> Calle 100 (6 stops, 14 min)
Ride time: 38 min | Transfers: 3 | Total cost: 53.0 (penalty 5.0 min per transfer)
```

Veredicto: correcto. 38 min + 3 x 5 min = 53.0.

F3. Origen igual a destino

```
$ python -m src.cli "Héroes" "Héroes"
You are already at Héroes.
```

Veredicto: correcto. Código de salida 0, sin buscar ruta.

F4. Entrada sin tildes y en distinta capitalización

```
$ python -m src.cli "heroes" "PORTAL AMERICAS"
Route: Héroes -> Portal Américas
Ride Caracas: Héroes -> Calle 26 (5 stops, 12 min)
Transfer at Calle 26: Caracas -> Calle 26
Ride Calle 26: Calle 26 -> Centro Memoria (1 stop, 2 min)
Transfer at Centro Memoria: Calle 26 -> NQS
Ride NQS: Centro Memoria -> Ricaurte (1 stop, 5 min)
Transfer at Ricaurte: NQS -> Americas
Ride Americas: Ricaurte -> Portal Américas (4 stops, 17 min)
Ride time: 36 min | Transfers: 3 | Total cost: 51.0 (penalty 5.0 min per transfer)
```

Veredicto: correcto. Los nombres se resuelven a la forma oficial (Héroes, Portal Américas).

F5. Estación desconocida con sugerencia

```
$ python -m src.cli "Calle 1000" "Ricaurte"
Unknown station: 'Calle 1000'. Did you mean: Calle 100, Calle 75, Calle 72?
```

Veredicto: correcto. Código de salida 1 y sugerencias útiles.

F6. --penalty 0 frente a la penalización por defecto

```
$ python -m src.cli "Portal Américas" "Calle 100" --penalty 0
Route: Portal Américas -> Calle 100
```

(mismos 4 tramos que F2)
Ride time: 38 min | Transfers: 3 | Total cost: 38.0 (penalty 0.0 min per transfer)

Penalización	Costo total	Ruta
5 (por defecto, F2)	53.0	3 transbordos
0	38.0	3 transbordos

Veredicto: correcto. El costo cambia (53.0 -> 38.0). La ruta es la misma porque en esta red reducida el trayecto óptimo no cambia con la penalización; se comprobó en los 380 pares (0 rutas distintas entre penalización 0 y 5).

F7. Modo interactivo (entrada por stdin)

```
$ printf 'Portal Américas\nCalle 100\n' | python -m src.cli
Origin station: Destination station: Route: Portal Américas -> Calle 100
(mismos 4 tramos y totales que F2, costo 53.0)
```

Veredicto: correcto. Los prompts no llevan salto de línea porque stdin no es una terminal.

F8. Penalización negativa

```
$ python -m src.cli "Ricaurte" "Calle 100" --penalty -1
usage: python -m src.cli [-h] [--list] [--penalty MINUTES] [--data DATA]
                        [origin] [destination]
python -m src.cli: error: argument --penalty: must be >= 0
```

Veredicto: correcto. Rechazado con código de salida 2.

Resumen

Caso	Escenario	Resultado
F1	Una línea	OK
F2	Varios transbordos	OK
F3	Misma estación	OK
F4	Tildes y mayúsculas	OK
F5	Estación desconocida	OK
F6	--penalty 0 vs defecto	OK
F7	Modo interactivo	OK
F8	Penalización negativa	OK

6. Verificación de optimalidad

Se comparó el costo de find_route (A*) con un Dijkstra independiente sobre estados (estación, línea), sin heurística, para todos los pares ordenados de estaciones (20 x 19).

```
pares comparados: 380
discrepancias: 0
```

Veredicto: A* devuelve el costo óptimo en los 380 pares. La misma comprobación está automatizada en test_optimal_cost_for_all_pairs.

7. Trazabilidad Git

```
$ git log --graph --oneline --all
* 8affc1f merge: feat/cli into main
|\
| * 002120a feat(cli): add command-line route planner
|/
* f4d74c2 merge: feat/astar-search into main
```

```

|\
| * a20f57b feat(search): add A* route search with transfer penalty
|/
* 5a3f307 merge: feat/graph-builder into main
|\
| * d4b70a2 feat(graph): build transport graph from knowledge-base facts
| * e2abd99 feat(kb): expose station coordinates as location facts
|/
* b13f5c8 chore: ignore .atl tooling directory
* 3cbfd19 merge: feat/knowledge-base into main
|\
| * e769c58 refactor(kb): split load_knowledge_base into validation helpers
| * 8fab9b8 feat(kb): add logic inference engine and TransMilenio knowledge base
|/
* 97e4130 docs: add project overview and setup guide
* 0a53cb5 chore: initialize route planning project

```

La salida se capturó antes de integrar esta rama de documentación, por lo que no incluye su commit ni su merge.

Ramas de feature (todas integradas en main con --no-ff): feat/knowledge-base, feat/graph-builder, feat/astar-search, feat/cli. Esta documentación vive en docs/test-report.

8. Limitaciones

- **Datos aproximados.** Tiempos y coordenadas de data/network.json son estimaciones para uso académico, no datos oficiales.
- **Paradas intermedias omitidas.** Estaciones adyacentes en el archivo pueden saltarse estaciones reales; el tiempo cubre ese tramo. Por eso el conteo de paradas del CLI no coincide con el real.
- **Red reducida.** Solo 4 líneas (Caracas, Calle 26, NQS, Américas) y 20 estaciones. No incluye rutas zonales, alimentadoras ni otras troncales.
- **Sin validación oficial.** Antes de usar los resultados fuera del curso, deben contrastarse con el mapa oficial de TransMilenio.
- **Modelo de costo simple.** Penalización fija por transbordo; no considera horarios, frecuencias, esperas ni tarifas.
- **Un solo autor.** El proyecto fue desarrollado por una persona, sin revisión de código por terceros.